Тема 7: Массивы и строки

1. Основы массивов 3
1.1. Определение массивов 4
1.2. Элементы массива 4
1.3. Доступ к элементам массива 5
1.4. Среднее арифметическое элементов массива 5
1.5. Инициализация массива 6
1.6. Многомерные массивы 8
1.6.1. Определение многомерного массива 9
1.6.2. Доступ к элементам многомерного массива 10
1.6.3. Форматирование чисел 10
1.6.4. Инициализация многомерных массивов 10
1.7. Передача массивов в функции 11
1.7.1. Объявление функции с аргументами в виде массивов 12
1.7.2. Вызов функции с массивом в качестве аргумента 13
1.7.3. Определение функции с массивом в качестве аргумента 13
1.8. Массивы структур 13
1.9. Массивы как члены классов 15
2. Массивы объектов 18
2.1. Массивы интервалов 18
2.2. Границы массива 20
2.3. Доступ к объектам в массиве 20
2.4. Массивы карт 21
2.4.1. Графические символы 24
2.4.2. Колода карт 24
2.4.3. Случайные номера 24
3. Строки 24
3.1. Строковые переменные 25
3.1.1. Как избежать переполнения буфера 26
3.1.2. Строковые константы 27
3.1.3. Чтение внутренних пробелов 27
3.2. Считывание нескольких строк 28
3.3. Копирование строк 29
3.4. Копирование строк более простым способом 30
4. Массивы строк 30
4.1. Строки как члены классов 32

4.2. Определенные пользователем типы строк 33
4.3. Стандартный класс string языка C++ 35
4.4. Определение объектов класса string и присваивание им значений 35
4.5. Ввод/вывод для объектов класса string 37
4.6. Поиск объектов класса string 38
4.7. Модификация объектов класса string 38
4.8. Сравнение объектов класса string 40
4.9. Доступ к символам в объектах класса string 41
4.10. Другие методы класса string 42

В повседневной жизни мы обычно объединяем похожие объекты в группы. Мы покупаем горошек в банках и яйца в коробках. В языках программирования нам тоже необходимо группировать вместе данные одинакового типа. Основным механизмом, использующимся для этих целей в C++, является массив. Он может содержать от нескольких единиц данных до многих миллионов. Данные, сгруппированные в массиве, могут быть как основных типов, таких, как int или float, так и определенных пользователем типов, таких, как структуры или объекты.

Массивы похожи на структуры тем, что они тоже объединяют некоторое количество переменных в большой блок. Но в структуре обычно группируются переменные разных типов, а в массиве группируются однотипные данные. Более важное отличие в том, что к элементам структуры можно получить доступ по имени, а к элементам массива — по индексу. Использование индекса для каждого из элементов позволяет упростить доступ в случае большого количества элементов.

Массивы существуют почти в каждом языке программирования. Массивы языка C++ похожи на массивы других языков и идентичны массивам языка C.

В этой главе мы сначала рассмотрим массивы элементов основных типов данных, таких, как int и char. Затем мы рассмотрим массивы, использующиеся как члены классов, и массивы, содержащие в себе объекты. Таким образом, в этой главе мы планируем не только знакомство с массивами, но и расширение ваших знаний об ООП.

В этой главе мы также рассмотрим два различных подхода к определению строк, которые используются для хранения текста и выполнения действий с ним. Первым видом строк является массив элементов типа char. Второй тип представляет собой объекты стандартного класса string в C++.

1. Основы массивов

Мы познакомимся с массивами на примере простой программы REPLAY. В ней создается массив, состоящий из четырех целых чисел, предназначенный для хранения данных о возрасте некоторых людей. Программа запрашивает ввод пользователем четырех значений, которые будут помещены в этот массив. В конце программы эти значения выводятся на дисплей.

```
// replay.cpp
// запоминание и вывод на экран информации, введенной пользователем
#include <iostream>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
  int age[4];
      for(int j = 0; j < 4; j++)
      {

    cout << "Введите возраст: ";
    cin >> age[j];
      }
  for(j = 0; j < 4; j++)
          cout << "Вы ввели: " << age[j] << endl;
      return 0;
}
```

Вот пример взаимодействия с программой:

Введите возраст: 44

Введите возраст: 16

Введите возраст: 23

Введите возраст: 68

Вы ввели: 44

Вы ввели: 16

Вы ввели: 23

Вы ввели: 68

В первом цикле for мы получаем значения возраста от пользователя и помещаем их в массив, затем, во втором цикле, мы считываем данные из массива и выводим их на дисплей.

1.1. Определение массивов

Как и другие переменные в C ++, массив должен быть определен перед его использованием. Как и другие определения, определение массива включает в себя тип хранящихся в нем переменных и имя массива. Но помимо этого для массива необходимо указать размер, который определяет, сколько элементов массив может содержать. Размер следует за именем и заключается в квадратные скобки. На рис. 7.1 показан синтаксис определения массива.

В примере REPLAY использован массив типа int. За типом следует имя массива, затем открывается квадратная скобка, указывается размер массива и наконец скобка закрывается. Значение в скобках должно быть выражением, определяющим целую константу. В примере мы использовали значение 4.

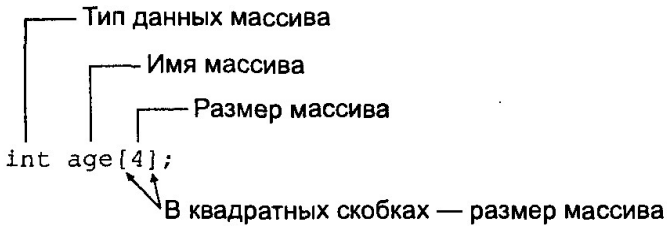


Рис. 7.1. Синтаксис определения массива

1.2. Элементы массива

Члены массива называют элементами. Как мы уже отмечали, все элементы массива одного типа, только значения у них разные. На рис. 7.2 показаны элементы массива age.

Согласно традиционному подходу, память на рисунке увеличивается вниз (хотя иногда изображают в обратном направлении). Поэтому первый элемент массива расположен в верхней части рисунка; следующие элементы располагаются ниже. Как записано в определении, массив может иметь только четыре элемента.

Заметим, что первый элемент массива имеет номер 0. Таким образом, так как всего в массиве четыре элемента, то последний элемент будет иметь номер 3. Эта ситуация иногда приводит к путанице: можно предположить, что последний элемент массива из четырех элементов должен иметь номер 4, но это не так.

1.3. Доступ к элементам массива

В примере REPLAY мы получаем доступ к элементам массива дважды. В первый раз — когда вставляем значение в массив в строке

```
cin >> age[j];
```

Во второй раз — когда мы считываем значения из массива в строке

```
cout << "Вы ввели: " << age[j] << endl;
```

В обоих случаях выражение для элемента массива будет таким:

```
age[j]
```

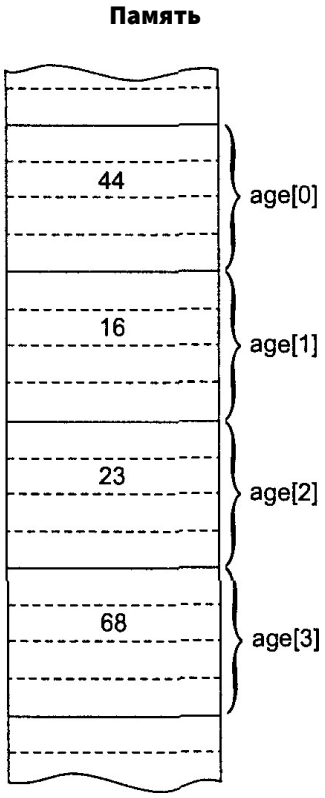


Рис. 7.2. Элементы массива

Оно состоит из имени массива и следующей за ним переменной j в квадратных скобках. Какой именно из элементов массива обозначает это выражение, зависит от значения j; age[0] ссылается на первый элемент, age[1] на второй, age[2] на третий, а age[3] на четвертый. Выражение, указывающееся в скобках, называется индексом массива.

Здесь j — переменная цикла для обоих циклов for, она увеличивается и принимает значения начиная с 0 и до 3; таким образом, мы последовательно получаем доступ к каждому из элементов массива.

1.4. Среднее арифметическое элементов массива

Вот другой пример работы массивов. В программе SALES пользователю предлагается ввести серию из шести значений, представляющих собой объемы продаж изделий за каждый день недели (исключая воскресенье), а затем программа вычисляет среднее арифметическое этих значений. Мы используем массив типа double, чтобы можно было пользоваться денежными значениями.

```
// sales.cpp
// определение среднего дневного объема продаж
#include <iostream>
```

```
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
    const int SIZE = 6;
    double sales[SIZE];
    cout << "Введите объем продаж на каждый из шести дней\n»;
    for(int j = 0; j < SIZE; j++)
        cin >> sales[j];
    double total = 0;
    for(j = 0; j < SIZE; j++)
        total += sales[j];
    double average = total / SIZE;
    cout << "Средний объем: " << average << endl;
    return 0;
}
```

Вот небольшой пример взаимодействия с программой SALES:

Введите объем продаж на каждый из шести дней

352.64

867.70

781.32

867.35

746.21

189.45

Средний объем: 643.11

Новая деталь в этой программе — это использование переменной константы для размера массива и в ограничениях цикла. Она определена в начале листинга:

```
const int SIZE = 6;
```

Использование переменной (вместо числа, такого, как 4, использованного в предыдущей программе) делает более простым изменение размера массива: нужно будет модифицировать только одну строку программы, чтобы изменился размер массива, ограничения циклов и значение в других местах, где встречается размер массива. Имя, написанное нами большими буквами, напоминает нам, что эта переменная не может быть изменена в программе.

1.5. Инициализация массива

Когда массив определен, мы можем присвоить его элементам значения. В примере DAYS 12 элементам массива days_per_month присваивается количество дней для каждого месяца.

```
// days.cpp
// показ количества дней с начала года и до введенной даты
#include <iostream>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
```

```
    int month, day, total_days;
    int days_per_month[12] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
    cout << "\nВведите месяц (от 1 до 12): ";
    cin >> month;
    cout << "\nВведите день (от 1 до 31): ";
    cin >> day;
    total_days = day;
    for(int j = 0; j < month - 1; j++)
        total_days += days_per_month[j];
    cout << "Общее число дней с начала года: " << total_days << endl;
    return 0;
}
```

Программа вычисляет количество дней от начала года до даты, определенной пользователем. (Внимание: программа не работает для високосных лет.) Вот пример взаимодействия с программой:

Введите месяц (от 1 до 12): 3

Введите день (от 1 до 31): 11

Общее число дней с начала года: 70

Получив значения дня и месяца, программа присваивает значение дня переменной totaLdays. Затем она проходит через цикл, в котором к переменной прибавляются значения элементов массива days_per_month. Количество этих значений на единицу меньше, чем значение месяца. Например, если пользователь ввел пятый месяц, то значения первых четырех элементов массива (31, 28, 31 и 30) будут прибавлены к переменной total_days.

Значения, которыми инициализируется массив days_per_month, заключаются в скобки и разделяются запятыми. Они связаны с определением массива знаком равенства. На рис. 7.3 показан синтаксис инициализации массива.

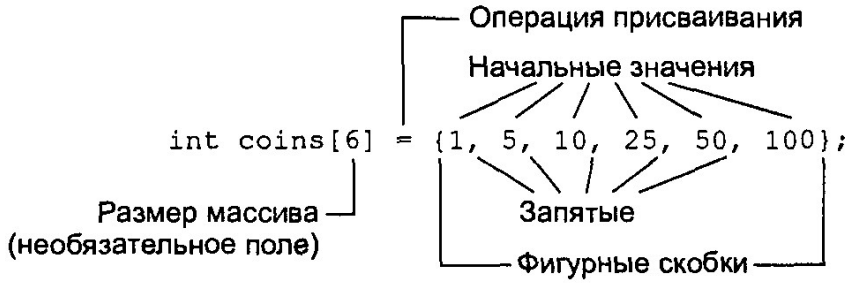


Рис. 7.3. Синтаксис инициализации массива

На самом деле нам не нужно использовать размер массива, когда мы инициализируем все элементы массива, так как компилятор может его вычислить, подсчитав инициализированные переменные. Таким образом, мы можем записать

```
int days_per_month[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
```

Что же случится, если вы явно использовали указание размера массива, но он не соответствует количеству элементов массива, которое инициализировано? Если инициализированы только несколько элементов, то оставшимся элементам будет присвоено значение 0. Если же инициализаций будет слишком много, то компилятор выдаст ошибку.

1.6. Многомерные массивы

До сих пор мы рассматривали одномерные массивы: одна переменная определяет каждый элемент массива. Но массивы могут иметь большую размерность. Вот программа SALEMON, в которой использован двумерный массив для хранения данных о продажах для нескольких отделов за несколько месяцев:

```
// salemon.cpp
// показ графика продаж
#include <iostream>
#include <iomanip>
using namespace std;
const int DISTRICTS = 4;
const int MONTHS = 3;
//////////////////////////////////////////////////////////
int main()
{
    int d, m;
    double sales[DISTRICTS][MONTHS];

    cout << endl;
    for(d = 0; d < DISTRICTS; d++)
        for(m = 0; m < MONTHS; m++)
        {
            cout << "Введите объем продаж для отдела " << d + 1;
            cout << ", за месяц " << m + 1 << ": ";
            cin >> sales[d][m];
        }
    cout << "\n\n";
    cout << " Месяц\n";
    cout << " 1         2          3 ";
    for(d = 0; d < DISTRICTS; d++)
    {
        cout << "\nОтдел " << d + 1;
        for(m = 0; m < MONTHS; m++)
            cout << setiosflags(ios::fixed)
                 << setiosflags(ios::showpoint) << setprecision(2)
                 << setw(10)
                 << sales[d][m];
    }
    cout << endl;
    return 0;
}
```

Эта программа принимает от пользователя данные о продажах и затем выводит их в виде таблицы.

Введите объем продаж для отдела 1, за месяц 1: 3964.23

Введите объем продаж для отдела 1, за месяц 2: 4135.87

Введите объем продаж для отдела 1, за месяц 3: 4397.98

Введите объем продаж для отдела 2, за месяц 1: 867.75

введите объем продаж для отдела 2, за месяц 2: 923.59

введите объем продаж для отдела 2, за месяц 3: 1037.01

введите объем продаж для отдела 3, за месяц 1: 12.77

введите объем продаж для отдела 3, за месяц 2: 378.32

введите объем продаж для отдела 3, за месяц 3: 798.22

введите объем продаж для отдела 4, за месяц 1: 2983.53

введите объем продаж для отдела 4, за месяц 2: 3983.73

введите объем продаж для отдела 4, за месяц 3: 9494.98

		Месяц	
	1	*2*	*3*
Отдел 1	*1432.07*	*234.50*	*654.01*
Отдел 2	*322.00*	*13838.32*	*17589.88*
Отдел 3	*9328.34*	*934.00*	*4492.30*
Отдел 4	*12838.29*	*2332.63*	*32.93*

1.6.1. Определение многомерного массива

Массив определяется двумя размерами, которые заключены в квадратные скобки:

```
double sales[DISTRICTS][MONTHS];
```

Вы можете считать sales двумерным массивом, похожим на шахматную доску. Иными словами можно сказать, что этот массив является массивом массивов. Это массив элементов DISTRICTS, каждый из которых является массивом элементов MONTHS. На рис. 7.4 показано, как это выглядит.

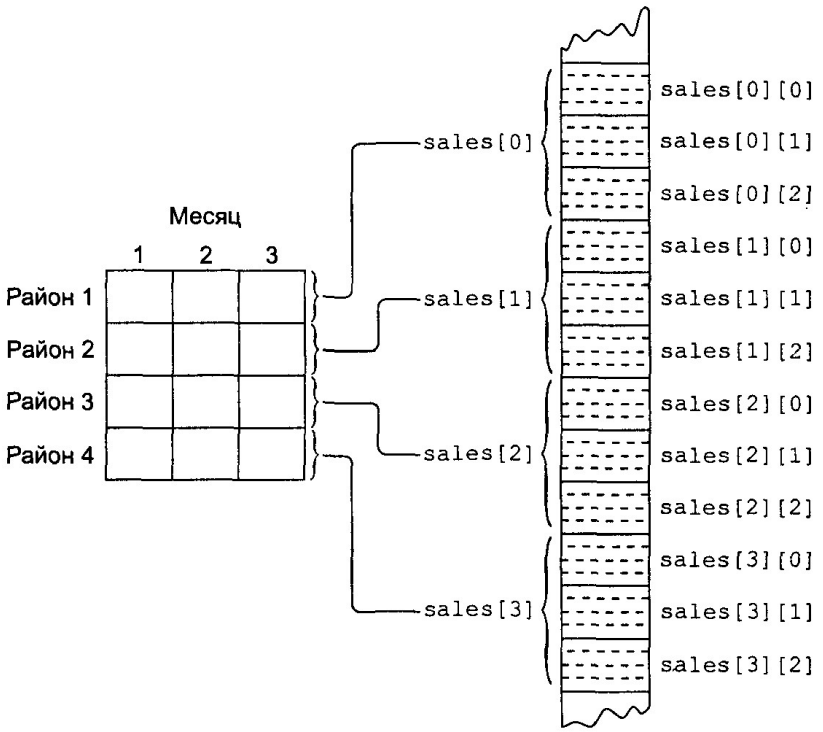


Рис. 7.4. Двумерный массив

Конечно, массивы могут иметь большую размерность. Трехмерный массив — это массив массивов, которые состоят из массивов. Доступ к элементам массива осуществляется с использованием трех индексов:

elem = dimen3[x][y][z];

Аналогично обстоит дело с одно- и двумерными массивами.

1.6.2. Доступ к элементам многомерного массива

Элементы двумерного массива требуют двух индексов:

```
sales[d][m];
```

Заметим, что каждый индекс заключается в отдельные квадратные скобки. Запятые не используются. Нельзя писать sales[d, m]; это работает в некоторых языках, но не в C++.

1.6.3. Форматирование чисел

Программа SALEMON выводит таблицу денежных значений. Важно, чтобы такие значения были правильно отформатированы. Давайте отступим от нашей темы и рассмотрим, как это делается в C++. Для денежных значений обычно используют два знака после запятой. Мы хотим выровнять значения в столбце. Также хорошо бы выводить последний ноль, то есть 79.50, а не 79.5.

Убедимся, что не потребуется много работы, чтобы реализовать это все с помощью потоков ввода/вывода C++. Вы уже видели, как использовался метод setw() для установки ширины выводимого поля. Форматирование десятичных чисел требует нескольких добавочных методов.

Вот строка, которая печатает числа с плавающей точкой, именованные как fpn, в поле шириной в 10 символов с двумя знаками после запятой:

```
cout << setiosflags(ios::fixed)      // нормальный неэкспоненциальный вид
<< setiosflags(ios::showpoint)       // всегда показывать десятичную точку
<< setprecision(2)                   // два знака после запятой
<< setw(10)                          // ширина вывода в 10 символов
<< fpn;                              // само число
```

Группа битовых форматирующих флагов в long int класса ios определяет, как должно быть выполнено форматирование. При этом нам не нужно знать, что такое класс ios или каковы причины использования существующего синтаксиса этого класса для осуществления работы выражений.

Мы коснемся только двух флагов: ios::fixed и ios::showpoint. Для их установки используйте метод setiosflags с именем флага в качестве аргумента. Имя должно предшествовать имени класса ios и отделяться от него операцией разрешения (::).

В первых двух строках оператора cout устанавливаются флаги ios (если вам нужно убрать флаги в более поздней точке программы, вы можете использовать метод resetiosflags). Флаг fixed предотвращает печать числа в экспоненциальной форме, например 3.45e3. Флаг showpoint определяет положение десятичной точки, даже если число не имеет дробной части: 123.00 вместо 123.

Для установки точности до двух знаков после запятой используйте метод setprecision с числом в качестве аргумента. Мы уже знаем, как установить ширину поля, используя метод setw. При установке всех этих параметров для cout вы сможете получить желаемый формат вывода числа.

1.6.4. Инициализация многомерных массивов

Как вы могли ожидать, вы можете инициализировать многомерные массивы. Только предварительно подготовьтесь к тому, что придется напечатать много скобок и запятых. Вот вариант программы SALEMON, которая использует инициализацию массива вместо запроса пользователя о вводе. Эта программа называется SALEINIT.

```
// saleinit.cpp
```

```
// показ графика продаж по данным массива
#include <iostream>
#include <iomanip>
using namespace std;
const int DISTRICTS = 4;      // размеры массива
const int MONTHS = 3;
//////////////////////////////////////////////////////////
int main()
{
    int d, m;
    double sales[DISTRICTS][MONTHS] =
    {
        {   1432.07,    234.50,     654.01      },
        {   322.00,     13838.32,   17589.88    },
        {   9328.34,    934.00,     4492.30     },
        {   12838.29,   2332.63,    32.93       }
    };
    cout << "\n\n";
    cout << " Месяц\n";
    cout << " 1        2          3 ";
    for(d = 0; d < DISTRICTS; d++)
    {
        cout << "\nОтдел " << d + 1;
        for(m = 0; m < MONTHS; m++)
        {
            cout << setw(10) << setiosflags(ios::fixed)
                 << setiosflags(ios::showpoint) << setprecision(2)
                 << sales[d][m];
        }
    }
    cout << endl;
    return 0;
}
```

Напомним, что двумерный массив в действительности является массивом массивов. Формат инициализации такого массива базируется на этом факте. Инициализирующие значения для каждого подмассива заключены в скобки и разделены запятыми:

{ 1432.07, 234.50, 654.01 }

а затем все эти подмассивы, каждый из которых является элементом главного массива, также заключаются в скобки и разделяются запятыми, как мы видели в листинге.

1.7. Передача массивов в функции

Массивы могут быть использованы как аргументы функций. Примером может служить вариант программы SALEINIT, в котором массивы объемов продаж передаются в функцию, выводящую данные в виде таблицы. Взгляните на листинг программы SALEFUNC:

```
// salefunc.cpp
// передача массива в виде параметра
#include <iostream>
#include <iomanip>
```

```
using namespace std;

const int DISTRICTS = 4;
const int MONTHS = 3;

void display(double[DISTRICTS][MONTHS]);
//////////////////////////////////////////////////////////
int main()
{
    double sales[DISTRICTS][MONTHS] =
    {
        { 1432.07, 234.50, 654.01 },
        { 322.00, 13838.32, 17589.88 },
        { 9328.34, 934.00, 4492.30 },
        { 12838.29, 2332.63, 32.93 }
    };
    display(sales);
    cout << endl;
    return 0;
}
//////////////////////////////////////////////////////////
// display()
// функция для вывода на экран массива
void display(double funsales[DISTRICTS][MONTHS])
{
    int d, m;
    cout << "\n\n";
    cout << " Месяц\n";
    cout << " 1          2          3 ";
    for(d = 0; d < DISTRICTS; d++)
    {
        cout << "\nОтдел " << d + 1;
        for(m = 0; m < MONTHS; m++)
        {
            cout << setiosflags(ios::fixed) << setw(10)
                 << setiosflags(ios::showpoint) << setprecision(2)
                 << funsales[d][m];
        }
    }
}
```

1.7.1. Объявление функции с аргументами в виде массивов

В объявлениях функции массивы-аргументы представлены типом данных и размером. Вот объявление функции display();

```
void display(double[DISTRICTS][MONTHS]);
```

На самом деле здесь есть один необязательный элемент. Следующее объявление работает так же:

```
void display(double[][MONTHS]);
```

Почему функции не нужно значение первой размерности? Вспомним, что двумерный массив — это массив массивов. Функция сначала рассматривает аргумент как массив отделов. Ей не важно знать, сколько отделов, но нужно знать, насколько велики элементы, представляющие собой отделы, так как тогда она сможет вычислить, где находится каждый элемент (умножая количество байтов, приходящееся на один элемент, на индекс нужного элемента). Поэтому мы можем сообщить функции размер массива MONTHS, но не сообщать, сколько таких массивов находится в массиве DISTRICTS.

Отсюда следует, что если мы объявили функцию с одномерным массивом в качестве аргумента, то нам не нужно указывать размер массива:

```
void somefunc(int elem[]);
```

1.7.2. Вызов функции с массивом в качестве аргумента

При вызове функции в качестве аргумента используется только имя массива.

```
display(sales);
```

Это имя (в нашем случае sales) в действительности представляет собой адрес массива в памяти. Мы не будем объяснять адреса детально до главы 10 «Указатели», но рассмотрим несколько общих вопросов.

Использование адресов для массивов-аргументов похоже на использование аргумента ссылки, при котором значения элементов массива не дублируются (копируются) в функции. (Смотрите обсуждение ссылочных аргументов в главе 5 «Функции».) Вместо этого функция работает с оригинальным массивом, хотя ссылается на него, используя другое имя. Эта система используется для массивов, потому что они могут быть очень большими; дублирование массива для каждой вызывающей его функции может отнимать много времени и пространства в памяти.

Однако адрес — это не то же самое, что и ссылка. С именем массива не используется амперсанд (&) при объявлении функции. Пока мы не изучили указатели, примите на веру, что массивы передаются в функцию только с использованием их имени и что функция работает при этом с оригиналом массива, а не с его копией.

1.7.3. Определение функции с массивом в качестве аргумента

Определение функции выглядит следующим образом:

```
void display(double funsales[DISTRICTS][MONTHS])
```

При записи аргумента-массива используются тип его данных, имя массива и его размерности. Имя массива, используемое при определении функции (в нашем случае funsales), может отличаться от имени массива, который затем будет использоваться в функции (sales), но оба этих имени ссылаются на один и тот же массив. Должны быть определены все размерности массива (исключая первую в некоторых случаях); они нужны функции для правильного обращения к элементам массива.

Обращаясь к элементам массива, функция использует то имя массива, которое было использовано при ее определении:

```
funsales[d][m];
```

При всех остальных операциях функции с массивом она действует так, как если бы массив был определен в самой функции.

1.8. Массивы структур

Массивы могут содержать в себе не только данные основных типов, но и структуры.

```
// partaray.cpp
// массив из структур
#include <iostream>
```

```
using namespace std;
const int SIZE = 4;
//////////////////////////////////////////////////////////
struct part
{
	int modelnumber;
	int partnumber;
	float cost;
};
//////////////////////////////////////////////////////////
int main()
{
	int n;
	part apart[SIZE];
	for(n = 0; n < SIZE; n++)
	{
		cout << endl;
		cout << "Введите номер модели: ";
		cin >> apart[n].modelnumber;
		cout << "Введите номер части: ";
		cin >> apart[n].partnumber;
		cout << "Введите стоимость: ";
		cin >> apart[n].cost;
	}
	cout << endl;
	for(n = 0; n < SIZE; n++)
	{
		cout << "Модель " << apart[n].modelnumber;
		cout << " Часть " << apart[n].partnumber;
		cout << " Стоимость " << apart[n].cost << endl;
	}
	return 0;
}
```

Пользователь вводит номер модели, номер части и стоимость части. Программа записывает эти данные в структуру. Однако эта структура является только одним из элементов массива структур. Программа запрашивает данные для разных частей и хранит их в четырех элементах массива apart. Затем она выводит информацию. Примеры вывода программы:

Введите номер модели: 44

Введите номер части: 4954

Введите стоимость: 133.45

Введите номер модели: 44

Введите номер части: 8431

Введите стоимость: 97.59

Введите номер модели: 77

Введите номер части: 9343

Введите стоимость: 109.99

Введите номер модели: 77

Введите номер части: 4297

Введите стоимость: 3456.55

Модель 44 Часть 4954 Стоимость 133.45

Модель 44 Часть 8431 Стоимость 97.59

Модель 77 Часть 9343 Стоимость 109.99

Модель 77 Часть 4297 Стоимость 3456.55

Массив структур определен в строке:

```
part apart[SIZE];
```

Здесь применен тот же синтаксис, что и для массивов, использующих основные типы данных. Только имя типа part показывает нам, что этот массив содержит данные более сложного типа.

Доступ к данным, членам структуры, которая является элементом массива, требует нового синтаксиса. Например,

```
apart[n].modelnumber
```

ссылается на переменную modelnumber структуры, которая является n элементом массива apart. На рис. 7.5 показано, как это выглядит.

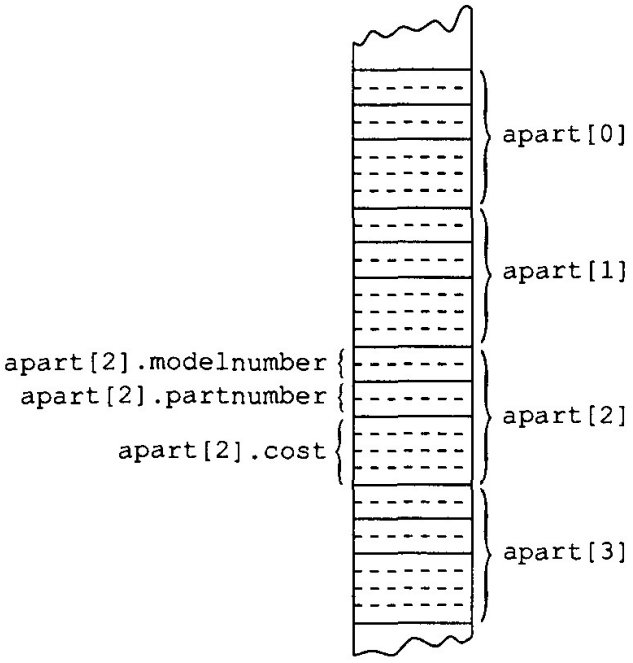


Рис. 7.5. Массив структур

Массивы структур — это полезный тип данных, используемый в различных ситуациях. Мы показали массив запчастей для машины, но мы могли хранить в массиве и личные данные сотрудников (имя, возраст, зарплата), и географические особенности городов (название, количество населения, высота над уровнем моря), и многое другое.

1.9. Массивы как члены классов

Массивы могут быть использованы в качестве полей класса. Давайте рассмотрим пример, который моделирует компьютерную структуру данных — стек.

Стек работает как пружинное устройство, удерживающее патроны в магазине. Когда вы помещаете патрон в магазин, стек немного оседает; когда вы вынимаете патрон, то он поднимается. Последний положенный в магазин патрон всегда будет вынут первым.

Стек — это один из краеугольных камней архитектуры микропроцессоров, используемой в самых современных компьютерах. Как мы упоминали ранее, функции передают аргументы и хранят возвращаемые адреса в стеке. Этот вид стека частично реализован в оборудовании, и к нему удобнее всего получать доступ через язык ассемблера. Однако стек можно полностью реализовать в программном обеспечении. Здесь стек представляет собой полезное устройство для хранения данных в определенной программной ситуации, например такой, как разбор алгебраического выражения.

В нашем примере, программе STAKARAY, создадим простой класс stack.

```
// stakaray.cpp
// класс стек
#include <iostream>
using namespace std;
////////////////////////////////////////////////////////////
class Stack
{
    private:
        enum { MAX = 10 };      // немного нестандартный синтаксис
        int st[MAX];            // стек в виде массива
        int top;                // вершина стека
    public:
        Stack()                 // конструктор
            { top = 0; }
        void push(int var)      // поместить в стек
            { st[++top] = var; }
        int pop()               // взять из стека
        { return st[top--]; }
};
////////////////////////////////////////////////////////////
int main()
{
    Stack s1;
    s1.push(11);
    s1.push(22);
    cout << "1: " << s1.pop() << endl;
    cout << "2: " << s1.pop() << endl;
    s1.push(33);
    s1.push(44);
    s1.push(55);
    s1.push(66);
    cout << "3: " << s1.pop() << endl;
    cout << "4: " << s1.pop() << endl;
    cout << "5: " << s1.pop() << endl;
    cout << "6: " << s1.pop() << endl;
    return 0;
}
```

Важным членом класса stack является массив st. Переменная top типа int хранит в себе индекс последнего элемента, положенного в стек; этот элемент располагается на вершине стека.

Размер массива, используемого для стека, определяется переменной MAX в строке

```
enum { MAX = 10 };
```

Это определение переменной MAX необычно. Придерживаясь философии инкапсуляции, предпочтительнее определять константы, используемые в классе, внутри класса — здесь так определена переменная MAX. Таким образом, использование глобальных констант-переменных для таких целей нецелесообразно. Стандарт C++ дает возможность объявления константы MAX внутри класса:

```
static const int MAX = 10;
```

Это означает, что MAX — константа и применяется для всех объектов класса. К сожалению, некоторые компиляторы не позволяют использовать эту конструкцию. Используя обходной путь, мы можем определить такие константы через перечисление (это описано в главе 4). Нам не нужно именовать само перечисление, нам нужен только один его элемент:

```
enum { MAX = 10 };
```

Здесь определена переменная MAX как целое со значением 10, определение дано внутри класса. Этот подход работает, но он неудобен. Если ваш компилятор поддерживает подход static const, то вам следует использовать его вместо определения констант внутри класса.

На рис. 7.6 показан стек. Так как на рисунке значения памяти возрастают по направлению сверху низ, то вершина стека находится в нижней части рисунка. При добавлении в стек нового элемента индекс переменной top увеличивается, и она будет показывать на новую вершину стека. При удалении элемента из стека индекс переменной top уменьшается. (Нам не нужно стирать старое значение из памяти при удалении элемента, оно просто становится несущественным.)



Рис. 7.6. Стек

Для помещения элемента в стек — процесс называемый проталкиванием — вы вызываете метод push() с сохраняемым значением в качестве аргумента. Для извлечения элемента из стека вы вызываете метод pop(), который возвращает значение элемента.

Программа main() примера STAKARAY применяет класс stack, создавая объект s1 этого класса. Программа помещает два элемента в стек, затем извлекает их и выводит на

дисплей. Затем в стек помещаются еще четыре элемента, далее они извлекаются и выводятся. Вот вывод программы:

```
1:22
2:11
3:66
4:55
5:44
6:33
```

Как вы можете видеть, элементы извлекаются из стека в обратном порядке; последний элемент, помещенный в стек, будет первым элементом, извлеченным оттуда.

Заметим небольшое отличие в использовании префиксной и постфиксной формы записи операторов увеличения и уменьшения. В строке

```
st[++top] = var;
```

метода push() сначала переменная top увеличивается и будет показывать уже на следующий элемент массива, расположенный за текущим последним элементом. Затем этому элементу присваивается значение переменной var, и он становится новой вершиной стека. В строке

```
return st[top--];
```

сначала возвращается значение вершины стека, а затем уменьшается переменная top, так, что она будет уже указывать на предыдущий элемент.

Класс stack является примером важной возможности ООП; он используется для реализации контейнера или механизма для хранения данных. Он применяется в связных списках, очередях, группах и т. д. Механизм хранения данных выбирается в соответствии с требованиями программы. Использование существующих классов для хранения данных означает, что программисту не нужно будет тратить время на их дублирование.

2. Массивы объектов

Мы видели, что объекты могут содержать массивы. Но верно и обратное: мы можем создать массив объектов. Рассмотрим две ситуации: массив интервалов и колоду карт.

2.1. Массивы интервалов

В главе 6 «Объекты и классы» мы показали несколько примеров класса Distance, который объединяет футы и дюймы в своем объекте, представляющем собой новый тип данных. Наша следующая программа ENGLARAY демонстрирует массив таких объектов.

```
// englaray.cpp
// объекты для английских мер
#include <iostream>
using namespace std;
////////////////////////////////////////////////////////////
class Distance
{
    private:
        int feet;
        float inches;
    public:
        void getdist()              // получение информации
```

```
        {
            cout << "\n Введите футы: "; cin >> feet;
            cout << " Введите дюймы: "; cin >> inches;
        }
        void showdist() const       // показ информации
        { cout << feet << "\'-" << inches << '\"';
};
//////////////////////////////////////////////////////////
int main()
{
    Distance dist[100];             // массив переменных
    int n = 0;                      // счетчик данных
    char ans;                       // ответ пользователя (y/n)
    cout << endl;

    do
    {
        cout << "Введите длину номер " << n + 1;
        dist[n++].getdist();        // получаем и сохраняем длину
        cout << "Продолжить ввод (y/n)?: ";
        cin >> ans;
    }
    while(ans != 'n');              // продолжать, пока не будет введено 'n'

    // показываем все введенное
    for(int j = 0; j < n; j++)
    {
        cout << "\nДлина номер " << j + 1 << " : ";
        dist[j].showdist();
    }
    cout << endl;
    return 0;
}
```

В этой программе пользователь может ввести произвольное количество интервалов. После введения каждого интервала программа спрашивает пользователя о том, нужно ли будет ввести еще один интервал. Если нет, то программа выводит все введенные ранее интервалы. Приведем пример взаимодействия с программой; здесь пользователь вводит три интервала:

Введите длину номер 1

Введите футы: 5

Введите дюймы: 4

Продолжить ввод (y/n)?: y

Введите длину номер 2

Введите футы: 6

Введите дюймы: 2.5

Продолжить ввод (y/n)?: y

Введите длину номер 3

Введите футы: 5

Введите дюймы: 10.75

Продолжить ввод (y/n)?: n

Длина номер 1 : 5'-4"

Длина номер 2 : 6'-2.5"

Длина номер 3 : 5'-10.75"

Конечно же, вместо простого вывода интервалов на дисплей программа может подсчитать их среднее значение, записать их на диск или выполнить какие-либо другие операции.

2.2. Границы массива

В этой программе ввод данных пользователем организован в виде цикла. Здесь пользователь может ввести сколько угодно структур типа part, даже больше значения MAX (равного здесь 100), представляющего собой размер массива.

Что случится, если пользователь введет более 100 интервалов? Что-то непредвиденное, но почти определенно плохое. В C++ нет проверки границ массива. Если программа поместит что-то за пределами массива, то компилятор и исполняемая программа протестовать не будут. Однако эти данные могут быть записаны поверх других данных или поверх самой программы. Это может послужить причиной странных эффектов или даже полного краха системы.

Мораль всего этого такова, что программисту следует применять проверку границ массива. Если возникает возможность переполнения массива данными, то он должен иметь большие размеры или у него должна быть возможность предупредить пользователя о возможном переполнении. Например, вы можете вставить этот код в начало цикла программы ENGLARAY:

```
if(n >= MAX)
{
    cout << "\nМассив полон!!!";
    break;
}
```

Таким образом вы сможете прервать цикл и предотвратить переполнение массива.

2.3. Доступ к объектам в массиве

Объявление класса Distance в этой программе похоже на то, которое использовалось в предыдущей программе. Однако в функции main() мы определили массив объектов класса Distance:

```
Distance dist[MAX];
```

Здесь типом данных массива dist является класс Distance. Число элементов массива определяется значением переменной MAX. На рис. 7.7 показано, как это выглядит.

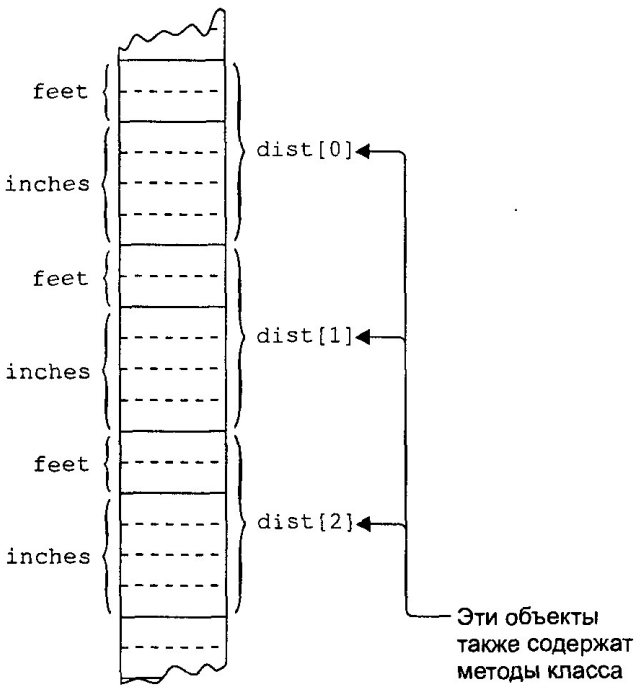


Рис. 7.7. Массив объектов

Доступ к методам объектов элементов массива похож на доступ к членам структуры, являющейся элементом массива, как в примере PARTARAY. Здесь показано, как вызвать метод showdist() элемента массива dist под номером j:

```
dist[j].showdist();
```

Как вы можете видеть, доступ к методу объекта, являющегося элементом массива, осуществляется с помощью операции точки. За именем массива следуют квадратные скобки, в которые заключен индекс элемента, затем операция точки и имя метода, за ними следуют скобки. Это похоже на доступ к полю структуры (или класса), за исключением того, что вместо имени переменной используется имя метода со скобками.

Заметим, что при вызове метода getdist() для помещения интервала в массив мы воспользуемся случаем и увеличим индекс массива n:

```
dist[n++].getdist();
```

Таким образом, следующая группа данных, полученная от пользователя, будет помещена в структуру следующего элемента массива dist. Переменная n должна быть увеличена вручную, как здесь, потому что мы используем цикл do вместо цикла for. В цикле for переменная цикла, которая увеличивается автоматически, может служить индексом массива.

2.4. Массивы карт

Вот другой, довольно длинный, пример массива объектов. Вы, конечно, помните пример CARDOBJ из главы 6. Мы воспользуемся классом card из этого примера и определим массив из 52 объектов этого класса, создав таким образом колоду карт. Вот листинг программы CARDARAY:

```
// cardaray.cpp
// класс игральных карт
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;
```

```
enum Suit { clubs, diamonds, hearts, spades };

// от 2 до 10 обычные числа
const int jack = 11;
const int queen = 12;
const int king = 13;
const int ace = 14;
//////////////////////////////////////////////////////////////
class card
{
    private:
        int number;
        Suit suit;
    public:
        card()    // конструктор
            { }
        void set(int n, Suit s)    // установка значения
            { suit = s; number = n; }
        void display();    // показ карты
};
//////////////////////////////////////////////////////////////
void card::display()
{
    if(number >= 2 && number <= 10)
        cout << number;
    else
    {
        switch(number)
        {
            case jack: cout << 'J'; break;
            case queen: cout << 'Q'; break;
            case king: cout << 'K'; break;
            case ace: cout << 'A'; break;
        }
    }
    switch(suit)
    {
        case clubs: cout << static_cast<char>(5); break;
        case diamonds: cout << static_cast<char>(4); break;
        case hearts: cout << static_cast<char>(3); break;
        case spades: cout << static_cast<char>(6); break;
    }
//////////////////////////////////////////////////////////////
int main()
{
    card deck[52];
    int j;
    cout << endl;
    for(j = 0; j < 52; j++)    // создаем упорядоченную колоду карт
```

```
    {
        int num = (j % 13) + 2;
        Suit su = Suit(j / 13);
        deck[j].set(num, su);
    }
    // показываем исходную колоду
    cout << "Исходная колода:\n»;
    for(j = 0; j < 52; j++)
    {
        deck[j].display();
        cout << " ";
        if(!((j + 1) % 13))     // начинаем новую строку после каждой 13-й карты
            cout << endl;
    }
    srand(time(NULL));      // инициализируем генератор случайных чисел
    for(j = 0; j < 52; j++)
    {
        int k = rand() % 52;      // выбираем случайную карту
        card temp = deck[j];      // и меняем ее с текущей
        deck[j] = deck[k];
        deck[k] = temp;
    }
    // показываем перемешанную колоду
    cout << "\nПеремешанная колода:\n»;
    for(j = 0; j < 52; j++)
    {
        deck[j].display();
        cout << " ";
        if(!((j + 1) % 13))     // начинаем новую строку после каждой 13-й карты
            cout << endl;
    }
    return 0;
}
```

Теперь, раз уж мы создали колоду карт, нам, наверное, стоит перемешать их. В программе мы показали карты, перемешали их и показали вновь. Для экономии пространства мы использовали графические символы мастей: трефы, черви, пики и бубны. На рис. 7.8 показан вывод программы. В эту программу включены несколько новых идей, давайте рассмотрим их по очереди.

Упорядоченная колода

2♣ 3♣ 4♣ 5♣ 6♣ 7♣ 8♣ 9♣ 10♣ J♣ Q♣ K♣ A♣
2♦ 3♦ 4♦ 5♦ 6♦ 7♦ 8♦ 9♦ 10♦ J♦ Q♦ K♦ A♦
2♥ 3♥ 4♥ 5♥ 6♥ 7♥ 8♥ 9♥ 10♥ J♥ Q♥ K♥ A♥
2♠ 3♠ 4♠ 5♠ 6♠ 7♠ 8♠ 9♠ 10♠ J♠ Q♠ K♠ A♠

Смешанная колода

3♠ 9♦ 6♦ K♠ 8♥ 4♠ 7♦ 4♣ 3♦ 3♣ A♥ 2♦ 9♣
6♣ 7♣ 9♥ 8♠ Q♣ Q♦ 10♥ J♣ 6♥ 4♥ J♦ K♣ 5♠
3♥ J♠ 5♣ K♦ Q♥ 10♦ 8♦ 2♣ 6♠ A♠ 4♦ J♥ 8♣
10♠ 2♥ Q♠ 10♣ 5♦ A♣ K♥ 7♥ 5♥ A♦ 2♠ 9♠ 7♠

Рис. 7.8. Вывод программы CARDARAY

2.4.1. Графические символы

Здесь мы использовали несколько специальных графических символов из кодов ASCII (см. приложение A «Таблица ASCII», список кодов ASCII.) В методе display() класса card мы использовали коды 5, 4, 3 и 6 для получения символов треф, бубен, червей и пик соответственно. Преобразуя эти номера к типу char, как в следующей строке,

```
static_cast<char>(5)
```

мы сможем напечатать их в виде символов, а не номеров.

2.4.2. Колода карт

Массив обьектов, предназначенный для колоды карт, определен в строке

```
card deck[52];
```

Здесь создается массив, названный deck, содержащий 52 объекта типа card. Для вывода j карты колоды мы вызовем метод display();

```
deck[j].display();
```

2.4.3. Случайные номера

Всегда весело, но иногда даже полезно генерировать случайные числа. В этой программе мы используем их для перемешивания колоды. Для получения случайных чисел необходимы два шага. Сначала генератор случайных чисел должен быть инициализирован, то есть должно быть задано начальное число. Для этого вызываем библиотечную функцию srand(). Эта функция использует в качестве начального числа системное время. Она требует двух заголовочных файлов: CSTDLIB и CTIME.

Здесь для генерации случайного числа мы вызвали библиотечную функцию rand(). Она возвращает случайное целое число. Для получения числа из диапазона от 0 до 51 мы применили операцию получения остатка от деления на 52 к результату функции rand().

```
int k = rand() % 52;
```

Получившееся случайное число k затем используется как индекс для обмена двух карт. Мы выполняем цикл for, меняя местами карту, чей индекс принадлежит картам от 0 до 51, с другой картой, чей индекс является случайным числом. Когда все 52 карты поменяются местами со случайной картой, колода будет считаться перемешанной. Эта программа может считаться основой для программ, реализующих карточные игры, но мы оставим детали для вас.

Массивы объектов широко используются в программировании на C++. Мы рассмотрим и другие примеры далее.

3. Строки

В начале этой главы мы заметили, что обычно в C++ используются два вида строк: строка как массив символов типа char и строка как объект класса string. В этом разделе мы опишем первый тип строк, который относится к нашей теме, так как строка — это массив элементов типа char. Мы называем его строковым, так как ранее он был единственным типом представления строк в C (и ранних версиях C++). Этот тип можно также назвать char*-строками, так как он может быть представлен в виде указателя на char.

Тем не менее строки, созданные с помощью класса string, который мы рассмотрим в следующем разделе, во многих ситуациях вытеснили строковый тип. Но строковый тип все еще используется по многим причинам. Во-первых, он используется во многих

библиотечных функциях языка С. Во-вторых, он год за годом продолжает появляться в кодах. И в-третьих, для изучающих С++ строковый тип наиболее примитивен и поэтому легко понимается на начальном уровне.

3.1. Строковые переменные

Как и другие типы данных, строки могут быть переменными и константами. Мы рассмотрим эти две сущности перед тем, как начать рассматривать более сложные строковые операции. Вот пример, в котором определена одна строковая переменная. (В этом разделе мы будем предполагать, что слово строка означает строковый тип.) Программа просит пользователя ввести строку и помещает эту строку в строковую переменную, а затем выводит эту строку на дисплей. Листинг программы STRINGIN:

```
// stringin.cpp
// простая переменная строка
#include <iostream>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
    const int MAX = 80;             // максимальный размер строки
    char str[MAX];                  // сама строка
    cout << "Введите строку: ";  // ввод строки
    cin >> str;
    cout << "Вы ввели: " << str << endl;        // вывод результата
    return 0;
}
```

Определение строковой переменной str выглядит похожим (да так оно и есть) на определение массива типа char:

```
char str[MAX];
```

Мы используем операцию >> для считывания строки с клавиатуры и помещения ее в строковую переменную str. Этой операции известно, как работать со строками: она понимает, что это массив символов. Если пользователь введет строку «Секретарь» (тот, кто копирует рукописи) в этой программе, то массив str будет выглядеть чем-то похожим на рис. 7.9.

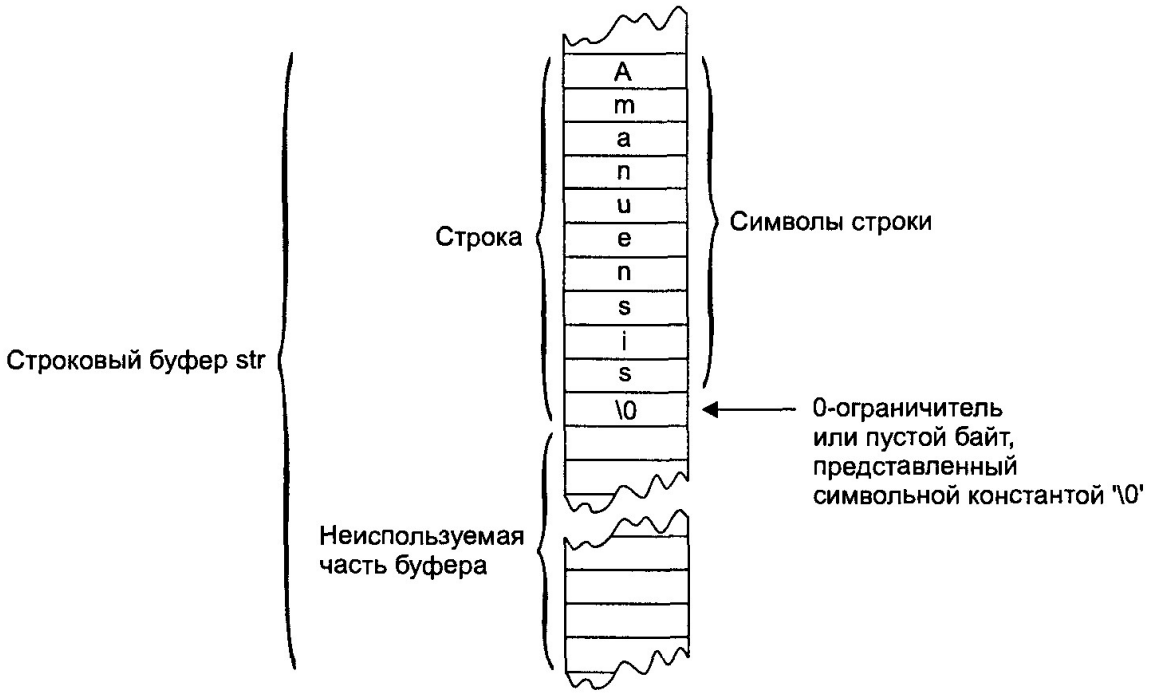


Рис. 7.9. Строка, хранящаяся в строковой переменной

Каждый символ занимает 1 байт памяти. Важная деталь, касающаяся строк, состоит в том, что они должны завершаться байтом, содержащим 0. Это часто представляют символьной константой \0, код которой в ASCII равен 0. Завершающий ноль называется нулевым символом. Когда операция **<<** выводит строку, то она выводит символы до тех пор, пока не встретит нулевой символ.

3.1.1. Как избежать переполнения буфера

Программа STRINGIN предлагает пользователю напечатать строку. Что случится, если пользователь введет строку, которая окажется длиннее, чем массив, используемый для ее хранения? Как мы упомянули ранее, в C++ нет встроенного механизма, защищающего программу от помещения элементов за пределы массива. Поэтому слишком увлеченный пользователь может привести систему к краху.

Однако существует возможность ограничить при использовании операции **>>** количество символов, помещаемых в массив. Программа SAFETYIN демонстрирует этот подход.

```
// safetyin.cpp
// избежание переполнения буфера
#include <iostream>
#include <iomanip>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
    const int MAX = 20;             // максимальный размер строки
    char str[MAX];                  // сама строка
    cout << "\nВведите строку: ";
    cin >> setw(MAX) >> str;        // ввод не более чем MAX символов
    cout << "Вы ввели: " << str << endl;
    return 0;
}
```

Эта программа использует метод setw, определяющий максимальное количество символов, которое сможет принять буфер. Пользователь может напечатать больше символов, но операция >> не вставит их в массив. В действительности операция вставит в массив на один символ меньше, чем определено, так как в буфере есть место для завершающего нулевого символа. Таким образом, в программе SAFETYIN максимальным числом возможных символов будет 19.

3.1.2. Строковые константы

Вы можете инициализировать строку постоянным значением при ее определении. Вот пример STRINIT, в котором это сделано (будем надеяться, что Александр Сергеевич не обидится на такое использование его стихов):

```
// strinit.cpp
// инициализация строки
#include <iostream>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
    char str[] = "Я памятник себе воздвиг нерукотворный.»;
    cout << str << endl;
    return 0;
}
```

Здесь строковая константа записана как нормальная фраза, ограниченная кавычками. Это может показаться необычным, так как строка — это массив типа char. В последнем примере мы видели, что массив инициализируется рядом значений, заключенных в скобки и разделенных запятыми. Почему переменная str не инициализирована так же? В действительности вы можете использовать такую последовательность символьных констант:

```
char str[] = { 'Я', ' ', ' п', 'а', ' м', 'я', 'т', 'н', 'и', 'к', ' ', 'с', 'е', 'б', 'е',
```

и т. д. К счастью, разработчики C++ (и C) сжалились над нами и предоставили нам упрощенный подход, показанный в программе STRINIT. Эффект тот же самый: символы помещаются один за другим в массив. Как во всех строках, последним символом будет нулевой.

3.1.3. Чтение внутренних пробелов

Если вы введете в программе STRINGIN строку, содержащую больше одного слова, то вы будете неприятно удивлены. Вот пример:

Введите строку: Идет бычок, качается, вздыхает на ходу.

Вы ввели: Идет

Куда же делся остаток фразы? Оказывается, что операция >> считает пробел нулевым символом. Таким образом, он считывает строки, состоящие из одного слова, и что-либо, напечатанное после пробела, отбрасывается.

Для считывания строк, содержащих пробелы, мы используем другой метод — cin.get(). Этот синтаксис означает использовать метод get() класса stream для его объекта cin. В следующем примере, BLANKSIN, показано, как это работает.

```
// blanksin.cpp
// ввод строки с пробелами
#include <iostream>
```

```
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
    const int MAX = 80;          // максимальная длина строки
    char str[MAX];               // сама строка
    cout << "\nВведите строку: ";
    cin.get(str, MAX);
    cout << "Вы ввели: " << str << endl;
    return 0;
}
```

Первый аргумент метода cin.get() — это адрес массива, куда будет помещена введенная строка. Второй аргумент определяет максимальный размер массива, автоматически предупреждая, таким образом, его переполнение.

При использовании этого метода введенная строка сохранится полностью.

Введите строку: Что посеешь, то и пожнешь.

Вы ввели: Что посеешь, то и пожнешь.

Проблемы могут возникнуть, если вы перепутаете метод cin.get() с операциями cin и >>. Мы обсудим использование метода ignore() класса cin для устранения этой проблемы в главе 12 «Потоки и файлы».

3.2. Считывание нескольких строк

Мы смогли решить проблему считывания внутренних пробелов в строке, но как быть с несколькими строками? Оказывается, что метод cin.get() может иметь третий аргумент, который пригодится нам в этой ситуации. Этот аргумент определяет символ, на котором метод завершает считывание строки. Установленным по умолчанию значением этого аргумента является символ новой строки ('\n'), но если вы вызовете метод с другим аргументом, то это значение заменится на введенный вами символ.

В нашем следующем примере, LINESIN, мы вызовем метод cin.get() с символом доллара ('$') в качестве третьего аргумента:

```
// linesin.cpp
// ввод нескольких строк
#include <iostream>
using namespace std;
const int MAX = 2000;        // максимальная длина строки
char str[MAX];               // сама строка
//////////////////////////////////////////////////////////
int main()
{
    cout << "\nВведите строку:\n";
    cin.get(str, MAX, '$');
    cout << "Вы ввели:\n" << str << endl;
    return 0;
}
```

Теперь вы можете напечатать столько строк для ввода, сколько хотите. Метод будет принимать символы до тех пор, пока вы не введете завершающий символ (или до тех пор, пока введенные данные не превысят размер массива). Помните, что вам нужно будет

нажать клавишу Enter после того, как вы напечатаете символ '$'. Вот пример взаимодействия с программой:

Введите строку:

Широка страна моя родная

Много в ней лесов, полей и рек

Я другой такой страны не знаю.

Где так вольно дышит человек.

$

Вы ввели:

Широка страна моя родная

Много в ней лесов, полей и рек

Я другой такой страны не знаю,

Где так вольно дышит человек.

Мы заканчивали каждую строку нажатием клавиши Enter, но программа продолжала принимать от нас ввод до тех пор, пока мы не ввели символ '$'.

3.3. Копирование строк

Лучшим способом понять истинную природу строк является произведение действий с ними символ за символом. Наша следующая программа делает именно это.

```
// strcopy1.cpp
// копирование строки с использованием цикла
#include <iostream>
#include <cstring>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
    char str1[] = "Маленькой елочке холодно зимой,";    // исходная строка
    const int MAX = 80;                                 // максимальная длина строки
    char str2[MAX];                                     // сама строка
    for(int j = 0; j < strlen(str1); j++)               // копируем strlen(str1)
        str2[j] = str1[j];                              // символов из str1 в str2
    str2[j] = '\0';                                     // завершаем строку нулем
    cout << str2 << endl;                               // и выводим на экран
    return 0;
}
```

Эта программа создает строковую константу str1 и строковую переменную str2. Затем в цикле for происходит копирование строковой константы в строковую переменную. За каждую итерацию цикла копируется один символ в строке программы

```
str2[j] = str1[j];
```

Вспомним, что компилятор объединяет две смежные строковые константы в одну, что позволяет нам записать цитату в двух строках.

Эта программа также знакомит нас с библиотечными функциями для строк. Так как в C++ нет встроенных операторов для работы со строками, то обычно используют эти функции. К счастью, их много. Одну из них мы использовали в программе strlen(), она определяет длину строки (сколько символов строка содержит). Мы используем длину строки для

ограничения цикла for, чтобы скопировать нужное нам количество символов. Для использования библиотечных функций для строк нужно подключить к программе заголовочный файл CSTRING (или STRING.H).

Скопированная версия строки должна заканчиваться нулевым символом. Однако длина строки, возвращаемая функцией strlen(), не включает в себя нулевой символ. Мы могли скопировать один добавочный символ, но проще вставить нулевой символ явно. Мы делаем это в строке

```
str2[j] = '\0':
```

Если вы не вставите этот символ, то затем увидите, что в строку, напечатанную программой, будут включены символы, следующие за нужной вам строкой. Операция << просто печатает все символы до тех пор, пока не встретит '\0'.

3.4. Копирование строк более простым способом

Конечно, вам не обязательно использовать цикл for для копирования строк. Как вы могли догадаться, для этого существует библиотечная функция. Приведем измененную версию предыдущей программы, STRCOPY2, которая использует функцию strcpy().

```
// strcopy2.cpp
// копирование строки функцией strcpy()
#include <iostream>
#include <cstring>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
    char str1[] = "Уронили мишку на пол, оторвали мишке лапу!»;
    const int MAX = 80;          // максимальная длина строки
    char str2[MAX];              // сама строка
    strcpy(str2, str1);          // копируем строку
    cout << str2 << endl;        // и показываем результат
    return 0;
}
```

Заметим, что первым аргументом этой функции является строка, куда будут копироваться данные:

```
strcpy(destination, source);
```

Порядок записи справа налево напоминает формат обыкновенного присваивания: переменная справа копируется в переменную слева.

4. Массивы строк

Если возможно существование массива массивов, значит возможен и массив строк. На самом деле это довольно полезная конструкция. Рассмотрим пример STRARAY, в котором названия дней недели помещаются в массив:

```
// straray.cpp
// массив строк
#include <iostream>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
```

```
    const int DAYS = 7;             // количество строк в массиве
    const int MAX = 12;             // максимальная длина каждой из них
    char star[DAYS][MAX] =      // массив строк
    {
        "Понедельник", "Вторник", "Среда", «Четверг",
        "Пятница", "Суббота", «Воскресенье"
    };
    for(int j = 0; j < DAYS; j++)  // вывод всех строк на экран
        cout << star[j] << endl;
    return 0;
}
```

Программа печатает все строки массива:

Понедельник

Вторник

Среда

Четверг

Пятница

Суббота

Воскресенье

Так как строки — это массивы, то будет верным утверждение, что star — массив строк — в действительности является двумерным массивом. Его первая размерность, DAYS, определяет, сколько строк в массиве. Вторая размерность, MAX, определяет максимальную длину строк (11 символов для строки «воскресенье» плюс 12-й завершающий нулевой символ). На рисунке 7.10 показано, как это выглядит.

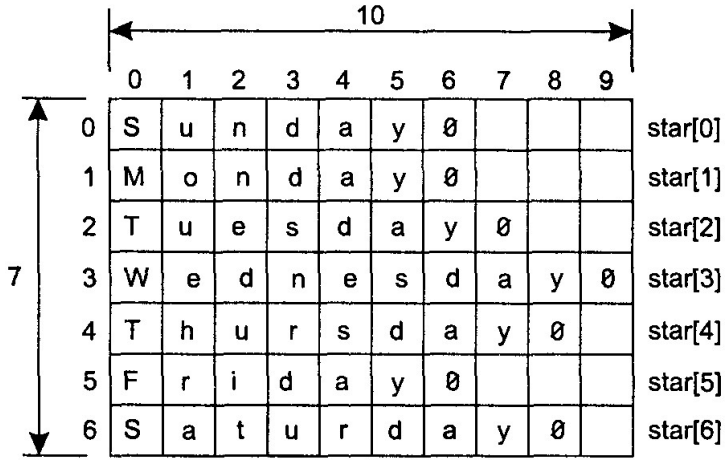


Рис. 7.10. Массив строк

Заметим, что некоторое количество памяти не будет использоваться в слу- чае, когда строка меньше максимальной длины. Мы объясним, как избежать этого недостатка, когда будем говорить об указателях.

Синтаксис для доступа к одной из строк массива может выглядеть немного неожиданно:

```
star[j];
```

Если мы работаем с двумерным массивом, то где тогда второй индекс? Так как двумерный массив — это массив массивов, то мы можем получить доступ к элементам «внешнего» массива, каждый из которых в отдельности является массивом (в нашем случае строкой).

Для того чтобы сделать это, нам не нужен второй индекс. Поэтому star[j] — это строка под номером j из массива строк.

4.1. Строки как члены классов

Строки часто используют в качестве членов классов. В следующем примере, вариации программы OBJPART из главы 6, используются строки, содержащие названия частей изделий.

```
// strpart.cpp
// использование строк в классе
#include <iostream>
#include <cstring>
using namespace std;
////////////////////////////////////////////////////////////
class part
{
  private:
    char partname[30];      // название
    int partnumber;       // номер
    double cost;       // цена
  public:
    void setpart(char pname[], int pn, double c)
    {
      strcpy(partname, pname);
      partnumber = pn;
      cost = c;
    }
    void showpart()
    {
      cout << "\nНазвание =" << partname;
      cout << ", номер =" << partnumber;
      cout << ", цена =$" << cost;
    }
};
////////////////////////////////////////////////////////////
int main()
{
  part part1, part2;
  part1.setpart("муфта", 4473, 217.55);
  part2.setpart("вороток", 9924, 419.25);
  cout << "\nПервая деталь: "; part1.showpart();
  cout << "\nВторая деталь: "; part2.showpart();
  cout << endl;
  return 0;
}
```

В этой программе определены два объекта класса part, и им присвоены значения в методе setpart(). Затем программа выводит эти объекты, используя метод showpart(). Приведем вывод:

Первая деталь:

Название = муфта, номер = 4473, цена =$217.55

Вторая деталь:

Название = вороток, номер = 9924, цена =$419.25

Чтобы уменьшить размер программы, мы удалили номер модели из переменных класса. В методе setpart() мы используем библиотечную функцию strcpy() для копирования строки из аргумента pname в переменную класса partname. Таким образом, этот метод служит для строковой переменной для тех же целей, что и операция присваивания для простых переменных. (Похожая функция strncpy() принимает еще третий аргумент, определяющий максимальное количество символов, которое может быть скопировано. Так можно предупредить переполнение массива.)

Кроме изученных нами, есть еще библиотечные функции для добавления одной строки к другой, сравнения строк, поиска определенного символа в строке и выполнения многих других операций. Описание этих функций вы сможете найти в документации к вашему компилятору.

4.2. Определенные пользователем типы строк

При использовании строкового типа в C++ иногда возникают проблемы. Одна из них заключается в том, что вы не можете использовать совершенно разумное выражение

```
strDest = strSrc;
```

для установки равенства одной строки другой. (В некоторых языках, таких, как BASIC, это замечательно работает.) В стандартном классе C++ string, который мы рассмотрим в следующем разделе, эта проблема решена, но на данный момент давайте посмотрим, как мы сможем использовать технологии ООП для самостоятельного решения этой проблемы. Создадим наш собственный класс string, что даст нам возможность представлять строки как объекты класса, который прольет свет на операции стандартного класса string.

Если мы определим наш собственный тип строк, используя класс, то мы сможем использовать выражения присваивания. (Многие другие операции со строками, такие, как объединение, могут быть также упрощены, но мы должны подождать до главы 8 «Перегрузка операций», чтобы увидеть, как это сделать.)

Программа STROBJ создает класс String. (Не путайте этот самопальный класс String со стандартным встроенным классом string, название которого начинается с маленькой буквы 's'.) Приведем листинг:

```
// strobj.cpp
// строка как класс
#include <iostream>
#include <cstring>
using namespace std;
////////////////////////////////////////////////////////////
class String
{
  private:
    enum { SZ = 80 };     // максимальный размер строки
    char str[SZ];     // сама строка
  public:
    String()      // конструктор без параметров
      { str[0] = '\0'; }
    String(char s[])      // конструктор с одним параметром
      { strcpy(str, s); }
```

```
    void display()      // вывод строки
      { cout << str; }
    void concat(String s2)      // сложение строк
    {
      if(strlen(str) + strlen(s2.str) < SZ)
        strcat(str, s2.str);
      else
        cout << "\nПереполнение!";
    }
};
//////////////////////////////////////////////////////////
int main()
{
  String s1("С Новым годом! "); // конструктор с одним параметром
  String s2 = "С новым счастьем!"; // аналогично, но в другой форме
  String s3; // конструктор без параметров

  cout << "\ns1 ="; s1.display();     // показываем все строки
  cout << "\ns2 ="; s2.display();
  cout << "\ns3 ="; s3.display();

  s3 = s1;      // присвоение
  cout << "\ns3 ="; s3.display();

  s3.concat(s2);      // сложение
  cout << "\ns3 ="; s3.display(); cout << endl;
  return 0;
}
```

Класс String содержит массив типа char. Может показаться, что наш заново определенный класс — это то же самое, что и определение строки: массив типа char. Но, поместив массив в класс, мы получили некоторую выгоду. Так как объекту может быть присвоено значение другого объекта того же класса, используя операцию =, то мы можем использовать выражения типа

```
s3 = s1;
```

как мы сделали в функции main(), для установки одного объекта класса String равным другому объекту. Мы можем также определить наши собственные методы для работы с объектами класса String.

В программе STROBJ все строки имеют одинаковую длину: SZ символов (SZ установлено равным 80). В ней есть два конструктора. В первом начальный символ в str устанавливается равным '\0', чтобы длина строки была равной 0. Этот конструктор вызывается в строке

```
String s3;
```

Второй конструктор присваивает объекту класса String значение строковой константы. Он использует библиотечную функцию strcpy() для копирования строковой константы в переменную объекта. Конструктор вызывается в строке

```
String s1("С Новым годом! ");
```

Альтернативный формат вызова этого конструктора, работающий с любым одноаргументным конструктором, будет таким

```
String s1 = "С Новым годом! ";
```
Какой бы формат вы ни использовали, этот конструктор эффективно преобразует строковый тип в объект класса String, то есть обычную строковую константу в объект. Метод display() выводит строку объекта на экран.

Другой метод нашего класса String, concat(), объединяет один объект класса String с другим (складывает две строки). Первоначальный объект вызывает метод, то есть к его строке будет добавлена строка объекта, переданного в качестве аргумента. Таким образом, строка функции main()

```
s3.concat(s2);
```
служит для прибавления строки s2 к уже существующей строке s3. Так как s2 инициализирована как «С новым счастьем!», а s3 было присвоено значение s1, «С Новым годом!», то в результате значением s3 будет «С Новым годом! С новым счастьем!»

Метод concat() использует библиотечную функцию языка С strcat() для выполнения объединения строк. Эта библиотечная функция присоединяет строку второго аргумента к строке первого аргумента. Вывод программы будет таким:

s1 = С Новым годом!

s2 = С новым счастьем!

s3= — *сейчас строка пуста*

s3= С Новым годом! — *аналог s1*

s3 = С Новым годом! С новым счастьем! — *после добавления s2*

Если длина двух строк, объединенных методом concat(), превысит максимальную длину строки для объекта String, то объединение не будет выполнено, а пользователь получит сообщение об ошибке.

Мы рассмотрели простой строковый класс. Теперь мы рассмотрим более сложную версию того же класса.

4.3. Стандартный класс string языка C++

Стандартный язык C++ включает в себя новый класс, называемый string. Этот класс во многом улучшает традиционный строковый тип. Например, вам не нужно заботиться о создании массива нужного размера для содержания строковых переменных. Класс string берет на себя всю ответственность за управление памятью. Кроме того, этот класс позволяет использовать перегруженные операции, поэтому вы можете объединять строковые объекты, используя операцию +:

```
s3 = s1 + s2;
```
Есть и другая выгода. Этот класс более эффективен и безопасен в использовании, чем строковый тип. Во многих ситуациях предпочтительнее использование класса. (Однако, как мы заметили ранее, существует и много ситуаций, в которых должен быть использован строковый тип.) В этом разделе мы рассмотрим класс string и его различные операции и методы.

4.4. Определение объектов класса string и присваивание им значений

Вы можете определить объект класса string различными способами: использовать конструктор без аргументов, создающий пустую строку, конструктор с одним аргументом, где аргумент является строковой константой, то есть символы ограничены двойными кавычками. Как и в написанном нами классе String, объекты класса string могут быть

присвоены один другому при использовании простой операции присваивания. В примере SSTRASS показано, как это выглядит.

```
// sstrass.cpp
// определение и присвоение для строк
#include <iostream>
#include <string>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
  string s1("Рыба");
  string s2 = "Мясо";
  string s3;
  s3 = s1;
  cout << "s3 >> " << s3 << endl;
  s3 = "Ни " + s1 + " ни ";
  s3 += s2;
  cout << "s3 >> " << s3 << endl;
  s1.swap(s2);
  cout << s1 << " не " << s2 << endl;
  return 0;
}
```

Здесь в первых трех строках кода показаны три способа определения объекта класса string. Первые два инициализируют строки, последний создает пустую переменную класса string. В следующей строке показано простое присваивание с использованием операции =.

Класс string использует перегруженные операции. Мы не будем изучать внутреннюю работу перегруженных операций до следующей главы, но можем использовать эти операции и без знания того, как они сконструированы.

Перегруженная операция + объединяет один строковый объект с другим. В строке

```
s3 = "Ни " + s1 + " ни ";
```

строка «Ни» помещается в переменную s3.

Вы можете также использовать операцию += для добавления строки в конец существующей строки. В строке

```
s3 += s2;
```

переменная s2, имеющая значение «Мясо», добавляется в конец строки s3. Получившаяся при этом строка «Ни Мясо» присваивается переменной s3.

В этом примере нам встретится первый метод класса string — swap(), который меняет местами значения двух строковых объектов. Он вызывается для одного объекта, в то время как другой объект является его аргументом. Мы применили этот метод к объектам s1(«Рыба») и s2(«Мясо»), а затем вывели значения этих объектов для того, чтобы показать, что s1 теперь имеет значение «Мясо», а s2 имеет значение «Рыба». Вот результат работы программы SSTRASS:

s3 = Рыба

s3 = Ни Рыба ни Мясо

Мясо не Рыба

4.5. Ввод/вывод для объектов класса string

Ввод и вывод осуществляются путем, схожим с применяемым для строкового типа. Операции << и >> перегружены для использования с объектами класса string, метод getline() принимает ввод, который может содержать пробелы или несколько строк. В примере SSTRIO показано, как это выглядит.

```
// sstrio.cpp
// ввод/вывод для класса string
#include <iostream>
#include <string>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
  string full_name, nickname, address;
  string greeting("Здравствуйте, ");

  cout << "Введите Ваше имя: ";
  getline(cin, full_name);
  cout << "Ваше имя: " << full_name << endl;

  cout << "Введите Ваш псевдоним: ";
  cin >> nickname;

  greeting += nickname;
  cout << greeting << endl;

  cout << "Введите Ваш адрес в несколько строк\n";
  cout << "Окончание ввода символ '$'\n";
  getline(cin, address, '$');
  cout << "Ваш адрес: " << address << endl;
  return 0;
}
```

Программа считывает имя пользователя, которое, возможно, содержит внутренние пробелы, используя метод getline(). Эта функция похожа на метод get(), используемый для строкового типа, но это не метод класса. Ее первым аргументом является потоковый объект, из которого будет приниматься ввод (здесь это cin), а второй аргумент — это объект класса string, куда будет помещен текст. Переменная full_name будет затем выведена, используя cout и операцию <<.

Затем программа считывает псевдоним пользователя, который предположительно состоит из одного слова, используя cin и операцию >>. Наконец, программа использует вариант функции getline() с тремя аргументами для считывания адреса пользователя, в котором может быть несколько строк. Третий аргумент функции определяет символ, который используется для завершения ввода. В программе мы использовали символ '$', который пользователь должен ввести, как последний символ строки перед тем, как нажать клавишу Enter. Если функция не имеет третьего аргумента, то предполагается, что завершающим символом строки будет '\n', представляющий клавишу Enter.

4.6. Поиск объектов класса string

Класс string включает в себя различные методы для поиска строк и фрагментов строк в объектах класса string. В программе SSTRFIND показаны некоторые из них.

```
// sstrfind.cpp
// поиск подстрок
#include <iostream>
#include <string>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
 string s1 = "В лесу родилась елочка, в лесу она росла.";
 int n;

 n = s1.find("елочка");
 cout << "Елочка найдена: " << n << endl;
 n = s1.find_first_of("умка");
 cout << "Первый из умка: " << n << endl;
 n = s1.find_first_not_of("абвгдАБВГД");
 cout << "Первый не из: " << n << endl;
 return 0;
}
```

Метод find() предназначен для поиска строки, используемой в качестве аргумента, в строке, для которой был вызван метод. Здесь мы ищем слово «елочка» в строке s1, которая содержит отрывок известной песенки. Оно найдено на позиции 16. Как в строковом типе, позиция самого левого символа нумеруется как 0.

Метод find_first_of() предназначен для поиска любого символа из группы и возвращает позицию первого найденного. Здесь ищем любой символ из группы 'у', 'м', 'к', 'а'. Первым из них будет найден символ 'у' в слове 'лесу' на позиции 5.

Похожий метод find_first_not_of() ищет первый символ в строке, который не входит в определенную группу символов. Здесь группа состоит из нескольких букв, прописных и строчных, поэтому функция находит первый символ пробела, который является вторым символом в строке. Вывод программы SSTRFIND будет таким:

Елочка найдена: 16

Первый из умка: 5

Первый не из абвгдАБВГД: 1

У многих из этих методов существуют варианты, которые мы не будем демонстрировать здесь, такие, как функция rfind(), переворачивающая строку, find_ last_of(), ищущая последний символ, совпадающий с группой заданных символов и find_last_not_of(). Все эти функции возвращают -1, если цель не найдена.

4.7. Модификация объектов класса string

Существуют различные пути модификации объектов класса string. В нашем следующем примере показаны методы erase(), replace() и insert() в работе.

```
// sstrchng.cpp
// изменение частей строки
#include <iostream>
```

```
#include <string>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
 string s1("Все хорошо, прекрасная маркиза.");
 string s2("принцесса");
 string s3("Приветствую ");

 s1.erase(0, 12);      // убираем "Все хорошо, "
 s1.replace(11, 7, s2);      // заменяем "маркиза" на "принцесса"
 s1.insert(0, s3);     // вставляем "Приветствую, "
 s1.erase(s1.size() - 1, 1);     // убираем '.'
 s1.append(3, '!');      // добавляем '!!!'

 int x = s1.find(' ');     // ищем пробелы
 while(x < s1.size())      // цикл по всем пробелам
 {
  s1.replace(x, 1, "/");      // заменяем пробел на '/'
  x = s1.find(' ');     // ищем следующий пробел
 }
 cout << "s1: " << s1 << endl;
 return 0;
}
```

Метод erase() удаляет фрагмент из строки. Его первым аргументом является позиция первого символа фрагмента, а вторым — длина фрагмента. В нашем примере удалено "Все хорошо," из начала строки. Метод replace() заменяет часть строки на другую строку. Его первым аргументом является позиция начала замены, вторым — количество символов исходной строки, которое должно быть заменено, а третьим аргументом является строка для замены. Здесь маркиза заменяется на принцесса.

Метод insert() вставляет строку, определенную во втором аргументе, на место, определенное в первом аргументе. В нашем примере "Приветствую" вставлено в начало строки s1. При втором использовании метода erase() применен метод size(), который возвращает количество символов в объекте класса string. Выражение size()-1 — это позиция последнего символа фрагмента, который будет удален. Метод append() ставит три восклицательных знака в конце предложения. В этой версии метода первый аргумент — это количество символов, которое будет добавлено, а второй аргумент — это символы, которые будут добавлены.

В конце программы мы показали идиому, которую вы можете использовать для перезаписи нескольких фрагментов в другой строке. Здесь в цикле while мы ищем символ пробела, используя метод find(), и заменяем каждый из них на слэш, используя функцию replace().

Мы начали с объекта s1, содержащего в себе строку «Все хорошо, прекрасная маркиза». После замены вывод программы SSTRCHNG будет таким:

s1: Приветствую/прекрасная/принцесса!!!

4.8. Сравнение объектов класса string

Можно использовать перегруженные операции или метод compare() для сравнения объектов класса string. Задача состоит в том, чтобы определить, являются ли строки идентичными или какая из них предшествует другой в алфавитном порядке. В программе SSTRCOM показаны некоторые возможности.

```
// sstrcom.cpp
// сравнение строк
#include <iostream>
#include <string>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
 string aName = "Иван";
 string userName;

 cout << "Введите Ваше имя: ";
 cin >> userName;
 if(userName == aName)
  cout << "Привет, Иван\n";
 else if(userName < aName)
  cout << "Ваше имя идет до имени Иван\n";
 else
  cout << "Ваше имя идет после имени Иван\n";

 int n = userName.compare(0, 2, aName, 0, 2);
 cout << "Первые две буквы Вашего имени ";
 if(n == 0)
  cout << "совпадают ";
 else if(n < 0)
  cout << "идут до ";
 else
  cout << "идут после ";
 cout << aName.substr(0, 2) << endl;
 return 0;
}
```

В первой части программы операции == и < используются для определения того, является ли написанное пользователем имя идентичным имени Иван или оно предшествует или следует за ним в алфавитном порядке. Во второй части программы метод compare() сравнивает только первые две буквы слова «Иван» с первыми двумя буквами имени, напечатанного пользователем (userName). Аргументами этой версии метода compare() являются начальная позиция userName, число символов, которые надо сравнить, строка, используемая для сравнения (aName), а также начальная позиция и количество символов в строке aName. Вот пример взаимодействия с программой SSTRCOM:

Введите Ваше имя: Алексей

Ваше имя идет до имени Иван

Первые две буквы Вашего имени идут до Ив

Первые две буквы имени «Иван» получают, используя метод substr(). Он возвращает фрагмент строки, для которой метод был вызван. Его первый аргумент — это позиция фрагмента, а второй — количество символов.

4.9. Доступ к символам в объектах класса string

Доступ к отдельным символам объектов класса string вы можете получить разными способами. В нашем следующем примере мы покажем доступ с использованием метода at(). Вы можете также использовать перегруженную операцию [], которая позволяет рассматривать объект класса string как массив. Однако операция [] не предупредит вас, если вы попытаетесь получить доступ к символу, лежащему за пределами массива (например, после конца строки). Операция [] ведет себя здесь так, как обращается с настоящим массивом, и это более эффективно. Но это может привести к тяжелым для диагностики программным ошибкам. Безопасней использовать метод at(), который остановит программу, если вы используете индекс, не входящий в допустимые границы. (Есть одно исключение, и мы обсудим его в главе 14 «Шаблоны и исключения».)

```
// sstrchar.cpp
// доступ к символам в строке
#include <iostream>
#include <string>
using namespace std;
//////////////////////////////////////////////////////////
int main()
{
 char charray[80];
 string word;
 cout << "Введите слово: ";
 cin >> word;
 int wlen = word.length();     // длина строки
 cout << "По одному символу: ";
 for(int j = 0; j < wlen; j++)
  cout << word.at(j);     // тут будет проверка на выход за пределы строки
 // cout << word[j];      // а тут проверки не будет
 word.copy(charray, wlen, 0);      // копируем строку в массив
 charray[wlen] = 0;
 cout << "\nМассив содержит: " << charray << endl;
 return 0;
}
```

В этой программе мы использовали метод at() для вывода содержимого объекта класса string символ за символом. Аргумент метода at() — это местонахождение символа в строке.

Затем мы показали, как вы можете использовать метод copy() для копирования объекта класса string в массив типа char, эффективно преобразовывая его в строковый тип. Вслед за копированием, после последнего символа строки дол- жен быть вставлен нулевой символ ('\0') для завершения преобразования к строковому типу. Метод length() класса string возвращает то же число, что и метод size(). Приведем вывод программы SSTRCHAR:

Введите слово: симбиоз

По одному символу: симбиоз

Массив содержит: симбиоз

(Вы можете также преобразовать объект класса string к строковому типу, используя методы c_str() или data(). Однако для использования этих методов вам нужно изучить указатели, которые мы рассмотрим в главе 10.)

4.10. Другие методы класса string

Мы видели, что методы size() и length() возвращают число символов строки объекта класса string. Количество памяти, занятое строкой, обычно больше, чем в действительности это необходимо. (Хотя если строка не была инициализирована, то она использует для символов 0 байтов.) Метод capacity() возвращает действительное количество занятой памяти. Вы можете добавлять символы в строку, не добавляя памяти в нее до тех пор, пока этот лимит не будет исчерпан. Метод max_size() возвращает максимально возможный размер объекта класса string. Это количество будет на три байта меньше, чем максимальное значение переменной типа int в вашей системе. В 32-битной Windows системе это 4 294 967 293 байта, но размер памяти будет, возможно, ограничен этим количеством.

Многие из методов класса string, которые мы обсудили, имеют варианты, отличающиеся количеством и типом аргументов. Обратитесь к документации вашего компилятора для выяснения деталей.

Вы должны знать, что объекты класса string не заканчиваются нулевым сим- волом, как это происходит в строковом типе. Вместо этого у нас есть переменная класса, хранящая длину строки. Поэтому, если вы будете просматривать строку, то не надейтесь найти в ней нулевой символ, обозначающий конец строки.

Класс string — в действительности только один из возможных строковых классов, производных от класса-шаблона basic_string. Класс string основан на типе char, но общий вариант использует тип wchar_t. Это позволяет классу basic_ string использоваться при работе с другими языками, имеющими больше символов, чем в русском. Файлы помощи вашего компилятора могут содержать спи- сок функций класса string в классе basic_string.